

Task AB13. Vision

 5 s  1024 MB

Le vaisseau spatial USS Enterprise (dont Sashka est le nouveau capitaine) parcourt l'univers, qui est une grille 2D infinie composée de secteurs. Dans chaque secteur (X, Y) ¹ se trouve une balise dont la portée est $V_{X,Y}$. Lorsqu'il se trouve dans ce secteur, le vaisseau utilise la balise pour balayer tous les secteurs (X', Y') tels que $X - V_{X,Y} \leq X' \leq X + V_{X,Y}$ et $Y - V_{X,Y} \leq Y' \leq Y + V_{X,Y}$. Ainsi, les secteurs visibles forment un carré de côté $2V_{X,Y} + 1$. Pour chacun de ces secteurs, la seule information visible est la portée de la balise qui s'y trouve, c'est-à-dire $V_{X',Y'}$ (car les balises agissent à la fois comme émetteurs et récepteurs). Après le balayage, le vaisseau se téléporte alors vers l'un de ces secteurs visibles, car les balises font également office de téléporteurs.

Malheureusement, les balises présentent un défaut majeur : elles peuvent inverser l'axe X du balayage, ou l'axe Y , ou les deux (c'est-à-dire qu'il existe 4 orientations possibles pour le balayage). Notez que le vaisseau utilise ce balayage potentiellement inversé pour décider vers quel secteur se téléporter, mais la téléportation utilise également la même orientation ; par exemple, si l'axe Y est inversé et que le vaisseau se téléporte vers la gauche (en diminuant Y) du balayage, il se téléportera en réalité vers la droite. Cela signifie que le vaisseau se téléportera bien vers le secteur choisi dans le balayage.

De plus, le vaisseau n'a pas de mémoire, donc sa décision quant à la destination de la téléportation doit dépendre uniquement du scan produit par la balise. Par ailleurs, la stratégie du vaisseau doit être déterministe : si on lui donne le même scan, il doit choisir de se téléporter vers le même secteur du scan.

Le vaisseau part de coordonnées inconnues et doit toujours (quel que soit son point de départ et quelles que soient les orientations des balayages qu'il reçoit) être capable de se déplacer dans la direction où X et Y augmentent, aussi loin que possible (on peut dire qu'on exige qu'il atteigne finalement un secteur (X, Y) tel que $X, Y \geq 10^{100}$). Cependant, pour que cela soit possible, la stratégie de déplacement du vaisseau et la répartition des puissances de vision à travers l'univers sont cruciales.

C'est là que vous intervenez : vous devez concevoir un motif rectangulaire P de dimensions N par M représentant les puissances de vision, qui se répétera à l'infini à travers l'univers : $V_{X,Y} = P_{X \bmod N, Y \bmod M}$. Vous devez également concevoir la stratégie de déplacement du vaisseau, c'est-à-dire une fonction qui prend en entrée le balayage produit par la balise et choisit vers lequel de ces secteurs se téléporter ensuite.

Comme les balises ayant une force de vision plus élevée sont coûteuses, votre objectif est de produire une solution valide, tout en essayant de minimiser la force de vision moyenne de votre motif, c'est-à-dire la valeur moyenne de P .

Comme ce problème peut sembler assez difficile, vous décidez de commencer par un problème d'entraînement : le même problème, mais en 1D. Dans ce problème plus simple, nous supprimons la dimension X et n'utilisons que la dimension Y . Ainsi, votre motif n'est qu'une séquence de longueur M et il se répète comme suit : $V_Y = P_{Y \bmod M}$. Les balayages produits par les balises ne sont également que des séquences de longueur $2V_Y + 1$ (contenant les secteurs Y' tels que $Y - V_{X,Y} \leq Y' \leq Y + V_{X,Y}$). Ici, il n'y a que 2 orientations possibles pour le balayage (régulier ou inversé) et le vaisseau doit pouvoir

¹ X est le numéro de ligne (croissant vers le bas) et Y est le numéro de colonne (croissant vers la droite).

atteindre $Y \geq 10^{100}$.

Vos solutions pour le cas 1D et le cas 2D seront notées séparément et vous pouvez obtenir des points pour l'un sans avoir de solution valide pour l'autre.

Détails de l'implémentation

Vous devez implémenter 4 fonctions : `getVisionPattern1d`, `getMove1d`, `getVisionPattern2d` et `getMove2d`. Les deux premières sont utilisées pour le cas 1D et les deux secondes pour le cas 2D. Dans chaque test, seule l'une de ces paires de fonctions sera appelée, mais les 4 doivent être implémentées (même si les implémentations d'une paire sont vides) pour que la solution soit valide (et puisse être compilée).

```
std::vector<int> getVisionPattern1d()
```

Cette fonction sera appelée une fois si le test concerne le cas 1D. Elle doit renvoyer une séquence P de longueur M — le motif de force de vision 1D à répéter. M et tous les éléments de P doivent être compris entre 1 et 60.

```
int getMove1d(std::vector<int> v)
```

Cette fonction sera appelée une fois par balayage possible, si le test porte sur le cas unidimensionnel. Elle reçoit un balayage produit par une balise, sous la forme d'une séquence de longueur $2V + 1$ (où V est la puissance de vision de la balise dans le secteur central de la séquence). Elle doit renvoyer le secteur vers lequel le vaisseau doit se téléporter, sous la forme d'un indice T dans la séquence, tel que $0 \leq T < 2V + 1$.

```
std::vector<std::vector<int>> getVisionPattern2d()
```

Cette fonction sera appelée une fois, si le test concerne le cas 2D. Elle doit renvoyer un tableau rectangulaire P de taille N par M — le motif de force de vision 2D à répéter. N , M et tous les éléments de P doivent être compris entre 1 et 60.

```
std::pair<int, int> getMove2d(std::vector<std::vector<int>> v)
```

Cette fonction sera appelée une fois par balayage possible, si le test porte sur le cas 2D. Elle reçoit un balayage produit par une balise, sous la forme d'un tableau carré de taille $2V + 1$ par $2V + 1$ (où V est la puissance de vision de la balise dans le secteur central du carré). Elle doit renvoyer le secteur vers lequel le vaisseau doit se téléporter, sous la forme d'une paire d'indices S et T dans le tableau, tels que $0 \leq S, T < 2V + 1$.

Pour la dimension D (1 ou 2) donnée du test, le correcteur vérifiera que votre programme produit un schéma valide — c'est-à-dire qu'il effectue des choix de téléportation valides pour tous les balayages possibles et que votre solution fonctionne indépendamment des coordonnées de départ et de la séquence d'orientations des balayages.

Contraintes

- $1 \leq D \leq 2$
- $1 \leq N, M, V_{X,Y}, V_Y \leq 60$

Sous-tâches et notation

Sous-tâche	Points	D	A^*
1	24	1	1, 5
2	76	2	1, 125

Votre score pour une sous-tâche donnée (si votre solution est correcte) dépend de la force de vision moyenne de votre motif P pour celle-ci. Soit A cette force et A^* la cible pour la sous-tâche. Votre score (fraction des points pour la sous-tâche) sera :

$$1 - \left(1 - \frac{A^* - 1}{\max(A, A^*) - 1} \right)^{0.8}$$

Exemple de motif

Supposons que votre solution renvoie le motif suivant avec $N = 3$ et $M = 2$:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

Examinons maintenant les balayages possibles au secteur $(0, 1)$, qui a une force de vision de 2. Il existe 4 orientations possibles (dont certaines produisent le même balayage sur ce motif) :

Orientation 0 : Pas de retournement

$$\begin{bmatrix} 4 & 3 & 4 & 3 & 4 \\ 6 & 5 & 6 & 5 & 6 \\ 2 & 1 & 2 & 1 & 2 \\ 4 & 3 & 4 & 3 & 4 \\ 6 & 5 & 6 & 5 & 6 \end{bmatrix}$$

Orientation 1 : Axe Y retourné

$$\begin{bmatrix} 4 & 3 & 4 & 3 & 4 \\ 6 & 5 & 6 & 5 & 6 \\ 2 & 1 & 2 & 1 & 2 \\ 4 & 3 & 4 & 3 & 4 \\ 6 & 5 & 6 & 5 & 6 \end{bmatrix}$$

Orientation 2 : axe X inversé

$$\begin{bmatrix} 6 & 5 & 6 & 5 & 6 \\ 4 & 3 & 4 & 3 & 4 \\ 2 & 1 & 2 & 1 & 2 \\ 6 & 5 & 6 & 5 & 6 \\ 4 & 3 & 4 & 3 & 4 \end{bmatrix}$$

Orientation 3 : les deux axes inversés

$$\begin{bmatrix} 6 & 5 & 6 & 5 & 6 \\ 4 & 3 & 4 & 3 & 4 \\ 2 & 1 & 2 & 1 & 2 \\ 6 & 5 & 6 & 5 & 6 \\ 4 & 3 & 4 & 3 & 4 \end{bmatrix}$$

Étant donné que 0 et 1 sont identiques et que le vaisseau est déterministe, un seul appel à `getMove2d` sera effectué pour eux. Supposons que votre solution renvoie le déplacement $(0, 1)$ lors de ce balayage. Pour l'orientation 0, cela signifierait se déplacer de 1 vers la gauche et de 2 vers le haut, tandis que pour l'orientation 1, cela signifierait se déplacer de 1 vers la droite et de 2 vers le haut.

Exemple de correcteur

La première entrée de l'exemple de correcteur est D . Ensuite, il effectuera tous les appels à votre fonction : il récupérera d'abord le motif P , puis mettra en cache tous les choix de téléportation. Après cela, il lancera une interaction en ligne de commande pour demander les coordonnées de départ du vaisseau. Il affichera ensuite l'état actuel : les coordonnées et le scan « brut » (sans inversions). Ensuite, il demandera l'orientation à utiliser (0 pour aucune rotation, 1 pour faire pivoter l'axe Y , 2 pour faire pivoter l'axe X et 3 pour faire pivoter les deux). Après cela, il affichera le scan dans l'orientation donnée, ainsi que le mouvement choisi par le vaisseau. Le vaisseau sera déplacé et ce processus se répétera. Notez que le correcteur d'exemple ne vérifiera pas automatiquement que votre solution est correcte.